



UNIVERSIDADE FEDERAL DE SERGIPE DEPARTAMENTO DE COMPUTAÇÃO-DCOMP

Docente: Dr. André Britto de Carvalho

Discentes:

Álex Santos Alencar

Filipe de Carvalho Godoy

Gabriel Ramos de Carvalho

João Andryel Santos Menezes

Paloma dos Santos

Trabalho Prático de Banco de Dados: Parte 2.1- Projeto Lógico-Modelo Relacional

Especificação do Projeto: Sistema de Gerenciamento de Banco de Sangue

1. Descrição Geral

O presente projeto tem como objetivo o desenvolvimento de um banco de dados destinado à gestão de um sistema de banco de sangue. A aplicação busca organizar e automatizar o processo de coleta, armazenamento, processamento e distribuição de bolsas de sangue e seus componentes, associando cada uma a um doador e a um possível receptor, considerando aspectos como tipo sanguíneo, validade da bolsa, disponibilidade e compatibilidade. O sistema também deve contribuir para a otimização dos estoques e a segurança dos processos envolvidos, bem como registrar todas as informações relacionadas aos usuários e profissionais envolvidos, permitindo dessa forma o acompanhamento do ciclo da doação à transfusão, garantindo segurança, rastreabilidade e eficiência no atendimento às demandas das instituições de saúde e dos seus pacientes.

2. Especificação

2.1. Descrição do sistema

O sistema de gerenciamento do banco de sangue tem como objetivo controlar de forma eficiente desde o cadastro a transfusão sanguínea. Ele é constituído de cadastro de doadores e receptores, registro de doações realizadas, controle do estoque de sangue disponível por tipo sanguíneo e outros requisitos, e ainda o atendimento de requisições feitas por hospitais. Além de possibilitar o rastreamento das doações, o monitoramento da validade do sangue em estoque, organiza os pedidos dos hospitais de acordo com a necessidade e

disponibilidade dos tipos sanguíneos, otimizando o atendimento contribuindo com a logística hospitalar.

2.2. Entidades e seus atributos

1. Entidade **Endereco** tem como atributos :id_endereco (chave primária-PK), pais, estado, cep, bairro, rua e cidade.
2. Entidade **Hemocentro** tem como atributos: id_hemocentro (chave primária-PK), nome_unidade e cnpj.
3. Entidade **Pessoa** trata-se de uma especialização (doador e receptor), tem como atributos: cpf (chave primária), telefone (multivalorado (1,n)), nome (composto (primeiro_nome, sobrenome), sexo, data_nascimento e idade (derivado).
4. Entidade **Hospital** tem como atributos: id_hospital (PK), nome_unidade e cnpj.
5. Entidade **Receptor** tem como atributos: cpf (FK).
6. Entidade **Transfusao** tem como atributos: id_transfusao (PK), data_transfusao, fator_rh e tipo_sanguineo.
7. Entidade **Medico** tem como atributos: id_medico (PK), especialidade, crm e nome.
8. Entidade **Enfermeiro** tem como atributos: id_enfermeiro (PK), cip, especialidade, turno e nome.
9. Entidade **Requisicao** tem como atributos: id_requisicao (PK), horário, data, e componente_solicitado.
10. Entidade **Coleta** tem como atributos: id_coleta (PK), data e hora.
11. Entidade **Amostra_sang** tem como atributos: id_amostra (PK), qualidade_amostra e data_coleta.
12. Entidade associativa **Vinculo** tem como atributos: id_hospital (Chave estrangeira (FK)), id_medico (FK) e turno.
13. Entidade **Banco_sangue** tem como atributos: id_banco_sang (PK), e capacidade_maxima.
14. Entidade **Exames** tem como atributos: id_exames (PK), data e status.
15. Entidade **Doacao** tem como atributos: id_doacao (PK) e data_doacao.
16. Entidade **Componente** tem como atributos: id_componentes (PK), volume, validade, tipo_sang, fator_rh e tipo_componente.
17. Entidade **Ponto_coleta** tem como atributos: id_ponto_coleta (PK) e tipo_unidade.

18. Entidade **Doador** tem como atributos: cpf (FK).
19. Entidade **Triagem** tem como atributos: id_triagem (PK), ultima_doação, peso, qtde-hemoglobina.
20. Entidade fraca **Testes** tem como atributos: id_amostra_sang (FK), id_componentes (FK), status e resultado

2.3. Relacionamentos

- Hemocentro **Possui** Endereço: relacionamento (1, 1) e (1, 1)
Cada Hemocentro possui exatamente 1 Endereço, e cada Endereço está vinculado (possui) a exatamente 1 Hemocentro.
- Hemocentro **Vinculado** Ponto_coleta: relacionamento (1, 1) e (0, n)
Cada Ponto_Coleta está vinculado a exatamente 1 Hemocentro, e um Hemocentro pode estar vinculado a nenhum, um ou vários Pontos_Coleta.
- Ponto_coleta **Realizada** Triagem: relacionamento (1, 1) e (0, n)
Cada Triagem é realizada em exatamente 1 Ponto_Coleta, e um Ponto de Coleta pode realizar nenhuma ou várias Triagens.
- Ponto_coleta **Possui** Endereço: relacionamento (1, 1) e (1, 1)
Cada Ponto_Coleta possui exatamente 1 Endereço, e cada Endereço possui exatamente 1 Ponto_Coleta.
- Hospital **Possui** Endereco: relacionamento (1, 1) e (1, 1)
Cada Hospital possui exatamente 1 Endereço, e cada Endereço possui exatamente 1 Hospital.
- Pessoa **Mora** Endereco: relacionamento (1, 1) e (1, 1)
Cada Pessoa Mora em exatamente 1 Endereço, e cada Endereço é ocupado (mora) por exatamente 1 Pessoa.
- Doador **Realiza** Triagem: relacionamento (1, 1) e (1, n)
Cada Doador realiza uma ou mais Triagens, e cada Triagem é realizada por exatamente 1 Doador.
- Triagem **Permite** Doacao: relacionamento (1, 1) e (0, 1)
Cada Doação está vinculada (Permite) a exatamente 1 Triagem, e uma Triagem permite nenhuma ou no máximo uma Doação.
- Doacao **Submetida** Exames: relacionamento (1, 1) e (1, 1)
Cada Doação é submetida a exatamente 1 Exame, e cada Exame está associado (submetido) a exatamente 1 Doação.
- Exames **Gera** Componentes: relacionamento (1, 1) e (0, n)
Cada Exame pode gerar nenhum, um ou vários Componentes, e cada Componente é gerado por exatamente 1 Exame

- Componentes **Submetido** Testes: relacionamento (1, 1) e (1, n)
Cada Componente é submetido a um ou mais Testes, e cada Teste está associado (submetido) a exatamente 1 Componente.
- Componentes **Armazenado** Banco_sangue: relacionamento (0, n) e (0, 1)
Cada Componente pode ser armazenado em nenhum ou 1 Banco de Sangue, e um Banco de Sangue pode armazenar nenhum ou vários Componentes.
- Componentes **Enviado** Transfusao: relacionamento (1, 1) e (1, 1)
Cada Componente é enviado para exatamente 1 Transfusão, e cada Transfusão utiliza exatamente 1 Componente.
- Transfusao **Recebe** Receptor: relacionamento (0, n) e (1, 1)
Cada Transfusão é destinada a exatamente 1 Receptor, e um Receptor pode receber nenhuma ou várias Transfusões.
- Hospital **Autoriza** Transfusao: relacionamento (1, 1) e (0, n)
Cada Transfusão é autorizada por exatamente 1 Hospital, e um Hospital pode autorizar nenhuma ou várias Transfusões.
- Transfusao **Depende** Requisicao: relacionamento (1, 1) e (1, 1)
Cada Transfusão depende de exatamente 1 Requisição, e cada Requisição está relacionada (Depende) a exatamente 1 Transfusão.
- Enfermeiro **Realiza** Transfusao: relacionamento (1, 1) e (0, n)
Cada Transfusão é realizada por exatamente 1 Enfermeiro, e um Enfermeiro pode realizar nenhuma ou várias Transfusões.
- Enfermeiro **Realiza** Coleta: relacionamento (1, 1) e (0, n)
Cada Coleta é realizada por exatamente 1 Enfermeiro, e um Enfermeiro pode realizar nenhuma ou várias Coletas.
- Coleta **Gera** Amostra_sang: relacionamento (1, 1) e (1, n)
Cada Coleta gera uma ou mais Amostras_Sang, e cada Amostra_Sang é gerada por exatamente 1 Coleta.
- Amostra_sang **Submetida** Testes: relacionamento (1, 1) e (1, n)
Cada Amostra-Sang é submetida a um ou mais Testes, e cada Teste está associado (Submetido) a exatamente 1 Amostra_Sang.
- Coleta **Submetido** Receptor: relacionamento (1, n) e (1, 1)
Cada Coleta está associada (Submetida) a exatamente 1 Receptor, e cada Receptor pode estar associado (Submetido) a uma ou mais Coletas.
- Receptor **Possui** Requisicao: relacionamento (1, 1) e (1, n)
Cada Receptor possui uma ou mais Requisições, e cada Requisição está associada a exatamente 1 Receptor.
- Medico **Emite** Requisicao: relacionamento (1, 1) e (0, n)

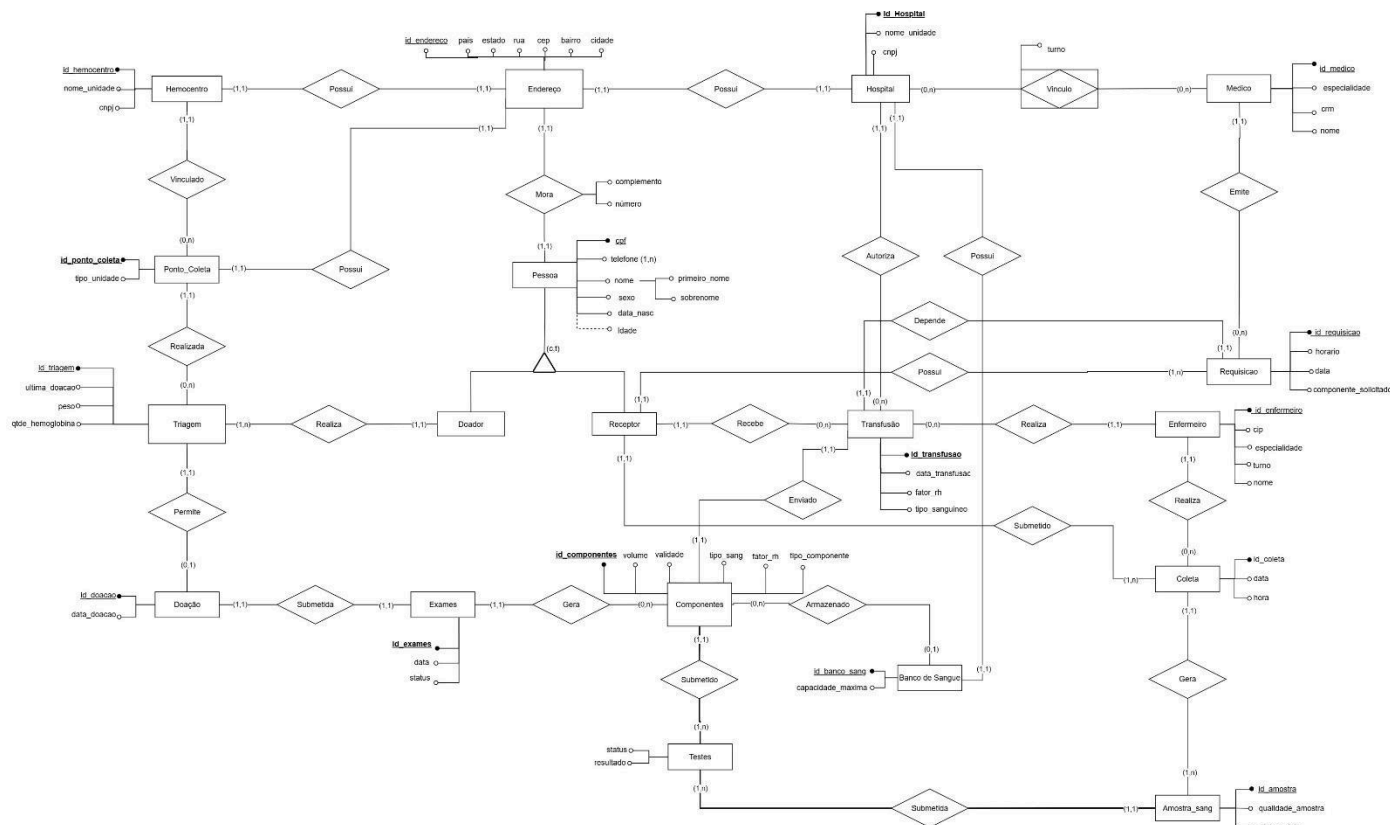
Cada Requisição é emitida por exatamente 1 Médico, e um Médico pode emitir nenhuma, uma ou várias Requisições.

- Médico **Vinculo** Hospital: relacionamento (0, n) e (0, n)
Um Médico pode estar vinculado a nenhum, um ou vários Hospitais, e um Hospital pode ter nenhum, um ou vários Médicos vinculados.
- Hospital **Possui** Banco_sangue: relacionamento (1, 1) e (1, 1)
Cada Hospital está associado (possui) a exatamente 1 Banco_Sangue, e cada Banco_Sangue possui exatamente 1 Hospital.

2.4. Diagrama Entidade-Relacionamento - DER

Na Figura 1 abaixo é apresentado o DER conforme mencionado nas sessões anteriores.

Figura 1- DER- Gerenciamento de Banco de Sangue.



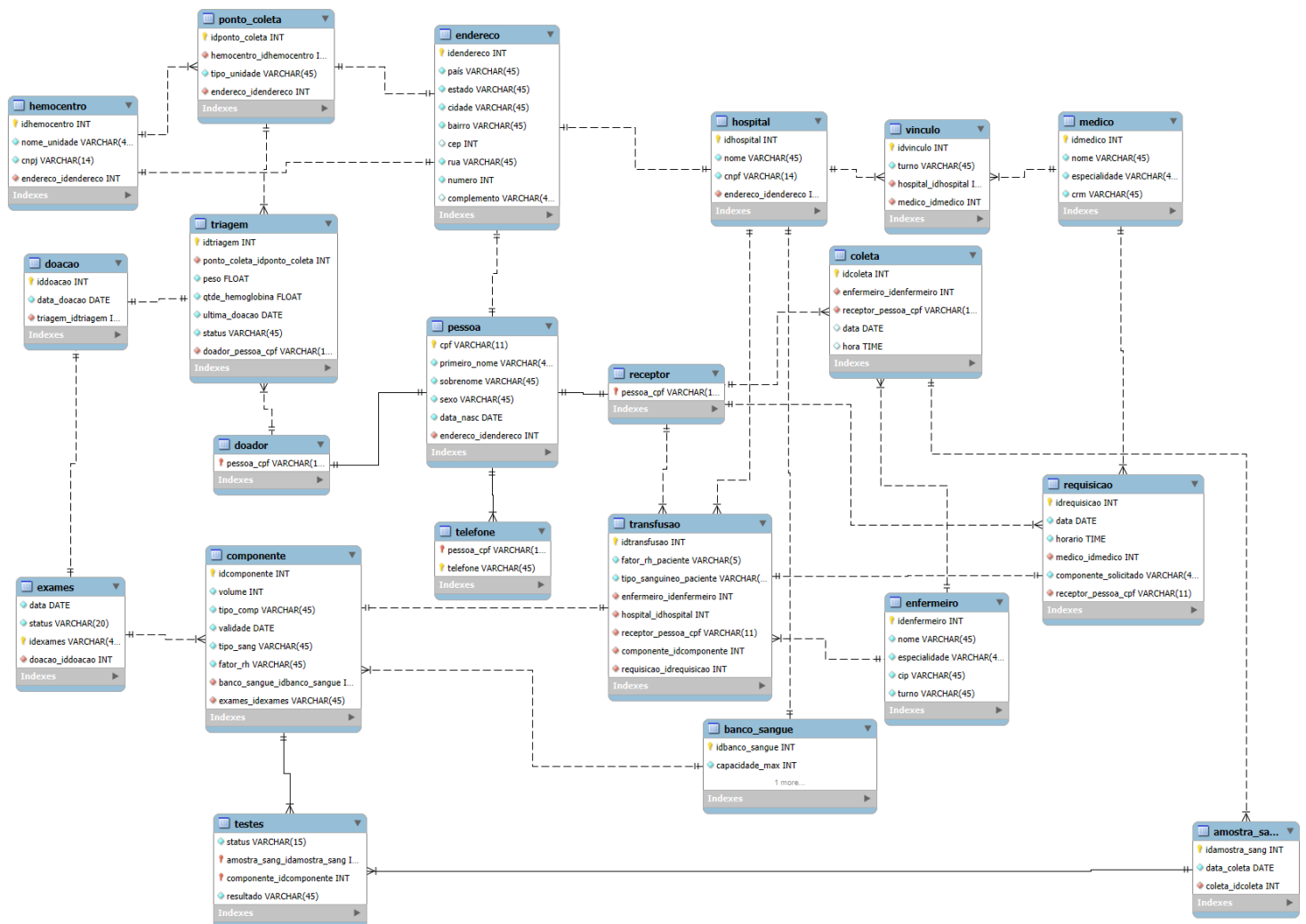
Fonte: Próprio Autor.

3. Mapeamento

3.1. Diagrama criado no *mysql-workbench*

As tabelas *mysql-workbench* são apresentadas na figura 2.

Figura 2- Tabelas *mysql-workbench* - Gerenciamento de Banco de Sangue.



Fonte: Próprio Autor.

3.2. Scripts Gerados

-- Schema mydb

CREATE SCHEMA IF NOT EXISTS mydb;

-- DOMAIN Definitions

```
-----  
  
CREATE DOMAIN mydb.DOM_STATUS_TRIAGEM AS VARCHAR(15)  
CHECK (VALUE IN ('Aprovado', 'Reprovado', 'Pendente'))  
DEFAULT 'Pendente';
```

```
  
CREATE DOMAIN mydb.DOM_TIPO_SANGUINEO AS VARCHAR(5)  
CHECK (VALUE IN ('A', 'B', 'AB', 'O'));
```

```
  
CREATE DOMAIN mydb.DOM_FATOR_RH AS VARCHAR(1)  
CHECK (VALUE IN ('+', '-'));
```

```
  
CREATE DOMAIN mydb.DOM_TURNIO AS VARCHAR(10)  
CHECK (VALUE IN ('Manhã', 'Tarde', 'Noite'));
```

```
-----  
-- Table mydb.endereco
```

```
-----  
CREATE TABLE IF NOT EXISTS mydb.endereco (  
    idendereco INTEGER NOT NULL,  
  
    país VARCHAR(45) NOT NULL DEFAULT 'Brasil',  
    estado VARCHAR(45) NOT NULL,  
    cidade VARCHAR(45) NOT NULL,  
    bairro VARCHAR(45) NOT NULL,  
    cep INTEGER NULL,  
    rua VARCHAR(45) NOT NULL,  
    numero INTEGER NOT NULL,  
    complemento VARCHAR(45) NULL,  
    PRIMARY KEY (idendereco)  
);
```

```
-----  
-- Table mydb.hemocentro
```

```
-----  
CREATE TABLE IF NOT EXISTS mydb.hemocentro (  
    idhemocentro INTEGER NOT NULL,  
    nome_unidade VARCHAR(45) NOT NULL,  
    cnpj VARCHAR(14) NOT NULL,  
    endereco_idendereco INTEGER NOT NULL,  
    PRIMARY KEY (idhemocentro),  
    CONSTRAINT cnpj_UNIQUE UNIQUE (cnpj),  
    CONSTRAINT fk_hemocentro_endereco1  
        FOREIGN KEY (endereco_idendereco)  
        REFERENCES mydb.endereco (idendereco)  
        ON DELETE NO ACTION  
        ON UPDATE NO ACTION  
);
```

```
  
CREATE INDEX fk_hemocentro_endereco1_idx ON mydb.hemocentro (endereco_idendereco);
```

```
-----  
-- Table mydb.pessoa
```

```

-----
CREATE TABLE IF NOT EXISTS mydb.pessoa (
  cpf VARCHAR(11) NOT NULL,
  primeiro_nome VARCHAR(45) NOT NULL,
  sobrenome VARCHAR(45) NOT NULL,

  sexo VARCHAR(45) NOT NULL CHECK (sexo IN ('Masculino', 'Feminino', 'Outro')),

  data_nasc DATE NOT NULL CHECK (data_nasc < CURRENT_DATE),
  endereco_idendereco INTEGER NOT NULL,
  PRIMARY KEY (cpf),
  CONSTRAINT fk_pessoa_endereco1
  FOREIGN KEY (endereco_idendereco)
  REFERENCES mydb.endereco (idendereco)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION
);

```

```

CREATE INDEX fk_pessoa_endereco1_idx ON mydb.pessoa (endereco_idendereco);

```

```

-----
-- Table mydb.hospital
-----
CREATE TABLE IF NOT EXISTS mydb.hospital (
  idhospital INTEGER NOT NULL,
  nome VARCHAR(45) NOT NULL,
  cnpf VARCHAR(14) NOT NULL,
  endereco_idendereco INTEGER NOT NULL,
  PRIMARY KEY (idhospital),
  CONSTRAINT cnpf_UNIQUE UNIQUE (cnpf),
  CONSTRAINT fk_hospital_endereco1
  FOREIGN KEY (endereco_idendereco)
  REFERENCES mydb.endereco (idendereco)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION
);

```

```

CREATE INDEX fk_hospital_endereco1_idx ON mydb.hospital (endereco_idendereco);

```

```

-----
-- Table mydb.receptor
-----
CREATE TABLE IF NOT EXISTS mydb.receptor (
  pessoa_cpf VARCHAR(11) NOT NULL,
  PRIMARY KEY (pessoa_cpf),
  CONSTRAINT fk_receptor_pessoa1
  FOREIGN KEY (pessoa_cpf)
  REFERENCES mydb.pessoa (cpf)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION
);

```

```

-----
-- Table mydb.enfermeiro
-----
CREATE TABLE IF NOT EXISTS mydb.enfermeiro (

```



```

idenfermeiro INTEGER NOT NULL,
nome VARCHAR(45) NOT NULL,
especialidade VARCHAR(45) NOT NULL,
cip VARCHAR(45) NOT NULL,
turno VARCHAR(45) NOT NULL,
PRIMARY KEY (idenfermeiro),
CONSTRAINT cip_UNIQUE UNIQUE (cip)
);

```

```

-----
-- Table mydb.banco_sangue
-----

```

```

CREATE TABLE IF NOT EXISTS mydb.banco_sangue (
  idbanco_sangue INTEGER NOT NULL,
  -- CHECK constraint: Garante que a capacidade seja um valor positivo.
  capacidade_max INTEGER NOT NULL CHECK (capacidade_max > 0),
  hospital_idhospital INTEGER NOT NULL,
  PRIMARY KEY (idbanco_sangue),
  CONSTRAINT fk_banco_sangue_hospital1
    FOREIGN KEY (hospital_idhospital)
    REFERENCES mydb.hospital (idhospital)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

```

```

CREATE INDEX fk_banco_sangue_hospital1_idx ON mydb.banco_sangue (hospital_idhospital);

```

```

-----
-- Table mydb.ponto_coleta
-----

```

```

CREATE TABLE IF NOT EXISTS mydb.ponto_coleta (
  idponto_coleta INTEGER NOT NULL,
  hemocentro_idhemocentro INTEGER NOT NULL,
  tipo_unidade VARCHAR(45) NOT NULL,
  endereco_idendereco INTEGER NOT NULL,
  PRIMARY KEY (idponto_coleta),
  CONSTRAINT fk_ponto_coleta_hemocentro1
    FOREIGN KEY (hemocentro_idhemocentro)
    REFERENCES mydb.hemocentro (idhemocentro)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT fk_ponto_coleta_endereco1
    FOREIGN KEY (endereco_idendereco)
    REFERENCES mydb.endereco (idendereco)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

```

```

CREATE INDEX fk_ponto_coleta_hemocentro1_idx ON mydb.ponto_coleta
(hemocentro_idhemocentro);
CREATE INDEX fk_ponto_coleta_endereco1_idx ON mydb.ponto_coleta (endereco_idendereco);

```

```

-----
-- Table mydb.doador
-----

```

```

CREATE TABLE IF NOT EXISTS mydb.doador (

```

```

    pessoa_cpf VARCHAR(11) NOT NULL,
    PRIMARY KEY (pessoa_cpf),
    CONSTRAINT fk_doador_pessoa1
    FOREIGN KEY (pessoa_cpf)
    REFERENCES mydb.pessoa (cpf)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

```

```

-----
-- Table mydb.triagem
-----

```

```

CREATE TABLE IF NOT EXISTS mydb.triagem (
    idtriagem INTEGER NOT NULL,
    ponto_coleta_idponto_coleta INTEGER NOT NULL,
    -- CHECK constraint: Garante que o peso do doador seja superior a 50.0.
    peso REAL NOT NULL CHECK (peso > 50.0),
    qtde_hemoglobina REAL NOT NULL,
    ultima_doacao DATE NOT NULL,
    -- DOMAIN usage: Utiliza o domínio criado para a coluna status.
    status mydb.DOM_STATUS_TRIAGEM NOT NULL,
    doador_pessoa_cpf VARCHAR(11) NOT NULL,
    PRIMARY KEY (idtriagem),
    CONSTRAINT fk_triagem_ponto_coleta1
    FOREIGN KEY (ponto_coleta_idponto_coleta)
    REFERENCES mydb.ponto_coleta (idponto_coleta)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
    CONSTRAINT fk_triagem_doador1
    FOREIGN KEY (doador_pessoa_cpf)
    REFERENCES mydb.doador (pessoa_cpf)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

```

```

CREATE INDEX fk_triagem_ponto_coleta1_idx ON mydb.triagem (ponto_coleta_idponto_coleta);
CREATE INDEX fk_triagem_doador1_idx ON mydb.triagem (doador_pessoa_cpf);

```

```

-----
-- Table mydb.doacao
-----

```

```

CREATE TABLE IF NOT EXISTS mydb.doacao (
    iddoacao INTEGER NOT NULL,
    -- DEFAULT constraint: Define a data atual como padrão para a doação.
    data_doacao DATE NOT NULL DEFAULT CURRENT_DATE,
    triagem_idtriagem INTEGER NOT NULL,
    PRIMARY KEY (iddoacao),
    CONSTRAINT fk_doacao_triagem1
    FOREIGN KEY (triagem_idtriagem)
    REFERENCES mydb.triagem (idtriagem)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

```

```

CREATE INDEX fk_doacao_triagem1_idx ON mydb.doacao (triagem_idtriagem);

```

```

-----
-- Table mydb.exames
-----
CREATE TABLE IF NOT EXISTS mydb.exames (
  idexames VARCHAR(45) NOT NULL,
  data DATE NOT NULL DEFAULT CURRENT_DATE,
  status VARCHAR(20) NOT NULL,
  doacao_iddoacao INTEGER NOT NULL,
  PRIMARY KEY (idexames),
  CONSTRAINT fk_processamento_doacao1
  FOREIGN KEY (doacao_iddoacao)
  REFERENCES mydb.doacao (iddoacao)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION
);

CREATE INDEX fk_processamento_doacao1_idx ON mydb.exames (doacao_iddoacao);

-----
-- Table mydb.componente
-----
CREATE TABLE IF NOT EXISTS mydb.componente (
  idcomponente INTEGER NOT NULL,
  -- CHECK constraint: Garante que o volume seja positivo.
  volume INTEGER NOT NULL CHECK (volume > 0),
  tipo_comp VARCHAR(45) NOT NULL,
  -- CHECK constraint: Garante que a data de validade seja no futuro.
  validade DATE NOT NULL CHECK (validade > CURRENT_DATE),
  -- DOMAIN usage: Utiliza os domínios para tipo sanguíneo e fator RH.
  tipo_sang mydb.DOM_TIPO_SANGUINEO NOT NULL,
  fator_rh mydb.DOM_FATOR_RH NOT NULL,
  banco_sangue_idbanco_sangue INTEGER NOT NULL,
  exames_idexames VARCHAR(45) NOT NULL,
  PRIMARY KEY (idcomponente),
  CONSTRAINT fk_componente_banco_sangue1
  FOREIGN KEY (banco_sangue_idbanco_sangue)
  REFERENCES mydb.banco_sangue (idbanco_sangue)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION,
  CONSTRAINT fk_componente_exames1
  FOREIGN KEY (exames_idexames)
  REFERENCES mydb.exames (idexames)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION
);

CREATE INDEX fk_componente_banco_sangue1_idx ON mydb.componente
(banco_sangue_idbanco_sangue);
CREATE INDEX fk_componente_exames1_idx ON mydb.componente (exames_idexames);

-----
-- Table mydb.medico
-----
CREATE TABLE IF NOT EXISTS mydb.medico (
  idmedico INTEGER NOT NULL,
  nome VARCHAR(45) NOT NULL,
  especialidade VARCHAR(45) NOT NULL,

```

```
    crm VARCHAR(45) NOT NULL,  
    PRIMARY KEY (idmedico)  
);
```

```
-----  
-- Table mydb.requisicao  
-----
```

```
CREATE TABLE IF NOT EXISTS mydb.requisicao (  
    idrequisicao INTEGER NOT NULL,  
    data DATE NOT NULL DEFAULT CURRENT_DATE,  
    horario TIME NOT NULL DEFAULT CURRENT_TIME,  
    medico_idmedico INTEGER NOT NULL,  
    componente_solicitado VARCHAR(45) NOT NULL,  
    receptor_pessoa_cpf VARCHAR(11) NOT NULL,  
    PRIMARY KEY (idrequisicao),  
    CONSTRAINT fk_requisicao_medico1  
        FOREIGN KEY (medico_idmedico)  
        REFERENCES mydb.medico (idmedico)  
        ON DELETE NO ACTION  
        ON UPDATE NO ACTION,  
    CONSTRAINT fk_requisicao_receptor1  
        FOREIGN KEY (receptor_pessoa_cpf)  
        REFERENCES mydb.receptor (pessoa_cpf)  
        ON DELETE NO ACTION  
        ON UPDATE NO ACTION  
);  
  
CREATE INDEX fk_requisicao_medico1_idx ON mydb.requisicao (medico_idmedico);  
CREATE INDEX fk_requisicao_receptor1_idx ON mydb.requisicao (receptor_pessoa_cpf);
```

```
-----  
-- Table mydb.transfusao  
-----
```

```
CREATE TABLE IF NOT EXISTS mydb.transfusao (  
    idtransfusao INTEGER NOT NULL,  
    fator_rh_paciente mydb.DOM_FATOR_RH NOT NULL,  
    tipo_sanguineo_paciente mydb.DOM_TIPO_SANGUINEO NOT NULL,  
    enfermeiro_idenfermeiro INTEGER NOT NULL,  
    hospital_idhospital INTEGER NOT NULL,  
    receptor_pessoa_cpf VARCHAR(11) NOT NULL,  
    componente_idcomponente INTEGER NOT NULL,  
    requisicao_idrequisicao INTEGER NOT NULL,  
    PRIMARY KEY (idtransfusao),  
    CONSTRAINT fk_transfusao_enfermeiro1  
        FOREIGN KEY (enfermeiro_idenfermeiro)  
        REFERENCES mydb.enfermeiro (idenfermeiro)  
        ON DELETE NO ACTION  
        ON UPDATE NO ACTION,  
    CONSTRAINT fk_transfusao_hospital1  
        FOREIGN KEY (hospital_idhospital)  
        REFERENCES mydb.hospital (idhospital)  
        ON DELETE NO ACTION  
        ON UPDATE NO ACTION,  
    CONSTRAINT fk_transfusao_receptor1  
        FOREIGN KEY (receptor_pessoa_cpf)  
        REFERENCES mydb.receptor (pessoa_cpf)  
        ON DELETE NO ACTION
```

```

        ON UPDATE NO ACTION,
CONSTRAINT fk_transfusao_componente1
    FOREIGN KEY (componente_idcomponente)
    REFERENCES mydb.componente (idcomponente)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
CONSTRAINT fk_transfusao_requisicao1
    FOREIGN KEY (requisicao_idrequisicao)
    REFERENCES mydb.requisicao (idrequisicao)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

CREATE INDEX fk_transfusao_enfermeiro1_idx ON mydb.transfusao (enfermeiro_idenfermeiro);
CREATE INDEX fk_transfusao_hospital1_idx ON mydb.transfusao (hospital_idhospital);
CREATE INDEX fk_transfusao_receptor1_idx ON mydb.transfusao (receptor_pessoa_cpf);
CREATE INDEX fk_transfusao_componente1_idx ON mydb.transfusao
(componente_idcomponente);
CREATE INDEX fk_transfusao_requisicao1_idx ON mydb.transfusao (requisicao_idrequisicao);

```

```

-----
-- Table mydb.coleta
-----
CREATE TABLE IF NOT EXISTS mydb.coleta (
    idcoleta INTEGER NOT NULL,
    enfermeiro_idenfermeiro INTEGER NOT NULL,
    receptor_pessoa_cpf VARCHAR(11) NOT NULL,
    data DATE NULL DEFAULT CURRENT_DATE,
    hora TIME NULL DEFAULT CURRENT_TIME,
    PRIMARY KEY (idcoleta),
    CONSTRAINT fk_coleta_enfermeiro1
        FOREIGN KEY (enfermeiro_idenfermeiro)
        REFERENCES mydb.enfermeiro (idenfermeiro)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION,
    CONSTRAINT fk_coleta_receptor1
        FOREIGN KEY (receptor_pessoa_cpf)
        REFERENCES mydb.receptor (pessoa_cpf)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION
);

CREATE INDEX fk_coleta_enfermeiro1_idx ON mydb.coleta (enfermeiro_idenfermeiro);
CREATE INDEX fk_coleta_receptor1_idx ON mydb.coleta (receptor_pessoa_cpf);

```

```

-----
-- Table mydb.amostra_sang
-----
CREATE TABLE IF NOT EXISTS mydb.amostra_sang (
    idamostra_sang INTEGER NOT NULL,
    data_coleta DATE NOT NULL DEFAULT CURRENT_DATE,
    coleta_idcoleta INTEGER NOT NULL,
    PRIMARY KEY (idamostra_sang),
    CONSTRAINT fk_amostra_sang_coleta1
        FOREIGN KEY (coleta_idcoleta)
        REFERENCES mydb.coleta (idcoleta)
        ON DELETE NO ACTION

```

```

    ON UPDATE NO ACTION
);

CREATE INDEX fk_amostra_sang_coleta1_idx ON mydb.amostra_sang (coleta_idcoleta);

-----
-- Table mydb.vinculo
-----
CREATE TABLE IF NOT EXISTS mydb.vinculo (
    idvinculo INTEGER NOT NULL,
    -- DOMAIN usage: Utiliza o domínio criado para a coluna turno.
    turno mydb.DOM_TURNO NOT NULL,
    hospital_idhospital INTEGER NOT NULL,
    medico_idmedico INTEGER NOT NULL,
    PRIMARY KEY (idvinculo),
    CONSTRAINT fk_vinculo_hospital1
        FOREIGN KEY (hospital_idhospital)
        REFERENCES mydb.hospital (idhospital)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION,
    CONSTRAINT fk_vinculo_medico1
        FOREIGN KEY (medico_idmedico)
        REFERENCES mydb.medico (idmedico)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION
);

CREATE INDEX fk_vinculo_hospital1_idx ON mydb.vinculo (hospital_idhospital);
CREATE INDEX fk_vinculo_medico1_idx ON mydb.vinculo (medico_idmedico);

-----
-- Table mydb.testes
-----
CREATE TABLE IF NOT EXISTS mydb.testes (
    status VARCHAR(15) NOT NULL,
    amostra_sang_idamostra_sang INTEGER NOT NULL,
    componente_idcomponente INTEGER NOT NULL,
    resultado VARCHAR(45) NOT NULL,
    PRIMARY KEY (amostra_sang_idamostra_sang, componente_idcomponente),
    CONSTRAINT fk_testes_amostra_sang1
        FOREIGN KEY (amostra_sang_idamostra_sang)
        REFERENCES mydb.amostra_sang (idamostra_sang)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION,
    CONSTRAINT fk_testes_componente1
        FOREIGN KEY (componente_idcomponente)
        REFERENCES mydb.componente (idcomponente)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION
);

CREATE INDEX fk_testes_componente1_idx ON mydb.testes (componente_idcomponente);

-----
-- Table mydb.telefone
-----

```

```
CREATE TABLE IF NOT EXISTS mydb.telefone (  
  pessoa_cpf VARCHAR(11) NOT NULL,  
  telefone VARCHAR(45) NOT NULL,  
  PRIMARY KEY (pessoa_cpf, telefone),  
  CONSTRAINT fk_telefone_pessoa1  
    FOREIGN KEY (pessoa_cpf)  
    REFERENCES mydb.pessoa (cpf)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION  
);
```

```
-----  
-- View mydb.view1  
-----
```

```
CREATE OR REPLACE VIEW mydb.view1 AS  
SELECT 1 AS id;
```

4. Referências

ELMASRI, R; NAVATHE, S.B. **Sistemas de Banco de Dados**, Addison Wesley, 6ª Edição.

Silberschatz, A; Korth H.F.; Sudarshan S. **Sistemas de Banco de Dados**, Editora Campus, 6a Edição.